Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Design Introduction

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

INDEX

1.3.1 Use Cases and Applications of Java in the Industry

Java is a **general-purpose, platform-independent language** that is widely adopted in many real-world applications. Its reliability, scalability, and rich ecosystem make it ideal for diverse industrial needs.

◈ Common Use Cases:

- **Web Applications**: Java powers dynamic, secure web apps using **Servlets**, **JSP**, **Spring Boot**, etc.
- **Enterprise Applications**: Java EE (now Jakarta EE) is a standard for large-scale, distributed enterprise systems (e.g., banking, insurance).
- **Android Mobile Apps**: Java is one of the primary languages for Android development.
- **Desktop Applications**: GUI-based apps using **JavaFX** or **Swing** (e.g., media players, admin panels).
- **Scientific Applications**: Used in simulations and data analysis tools (e.g., MATLAB uses Java internally).
- **Game Development**: Lightweight games and backend engines (e.g., Minecraft is built with Java).
- **Big Data Technologies**: Frameworks like **Hadoop** and **Apache Kafka** are built on Java.
- **Cloud-based Solutions**: Java is widely used in microservices and cloud platforms like AWS, Azure, and GCP.

◈ Java Platforms / Editions

1. **Java SE (Standard Edition)**
    - Core functionality for general-purpose programming.
    - Includes basic libraries: java.lang, java.util, java.io, etc.
    - Used to develop desktop and console apps.
2. **Java EE (Enterprise Edition)** *(now Jakarta EE)*
    - Builds on Java SE with libraries for enterprise features.
    - Supports web services, distributed systems, and components like Servlets, EJB.
3. **Java ME (Micro Edition)**
    - For small, resource-constrained devices like embedded systems, IoT.
    - Now mostly replaced by newer tech in mobile development.
4. **JavaFX**
    - A modern GUI toolkit for building rich desktop applications.
    - Successor to Swing with support for FXML and CSS-like styling.

1.3.2 Domains Where Java is Dominant

Domain	Java Role
Web Development	Backend development with frameworks like Spring, Hibernate.
Enterprise Solutions	Used in ERP, CRM, banking software, and telecom systems.
Mobile Development	Android apps primarily built using Java and Kotlin.
Embedded Systems	Java ME is used in smart cards, IoT, and embedded device software.
Cloud & SaaS	Java-based microservices and APIs deployed on cloud platforms.
Financial Services	Java is known for speed and security, essential for fintech and trading apps.

🔑 Why Java is Preferred in Industry:

- **Platform Independence**: Runs on any system with the JVM.
- **Strong Libraries & Frameworks**: Speeds up development (e.g., Spring, Hibernate).
- **Performance & Security**: Reliable for mission-critical systems.
- **Community & Support**: Massive developer community with constant updates and support.
- **Tooling & IDEs**: Excellent tools like IntelliJ, Eclipse, NetBeans.

1.4.1. Java Bytecode

Definition: Java bytecode is an intermediate, platform-independent code generated by the Java compiler (javac) after compiling .java files.

Each bytecode operation is represented by a single byte, which is why it's called "bytecode." This compact form of instruction allows for efficient execution.

➢How Is Bytecode Generated?

- When you write a Java program, you create source code (e.g.,.javafiles).
- The compiler converts this source code into bytecode (or machine code). This bytecode is stored in a .classfile.
- The Java Virtual Machine (JVM)then executes this bytecode.

➢Here's how it works:

- You write your program in a high-level language like Java.
- The compiler translates your Java source code into bytecode.
- The JVM (interpreter) executes this bytecode on any platform.

1.4.2. Compilation and Execution

When compiling a Java program, there are two main steps: Compilation and execution.

Let's break it down:

- **Compilation:**
  - **You write your Java code in a .javafile (also known as the source file).**
  - **The Java compiler (javac) processes this source code and converts it into bytecode.**
  - **Each class in your source file gets stored in a separate .classfile.**
  - **The bytecode is platform-independent and can run on any system.**
- **Execution:**
  - **The class files generated during compilation are machine-independent.**
  - **To run your program, you pass the main class file (the one containing the main method) to the Java Virtual Machine (JVM).**

Example Program (HelloWorld.java):

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello, World!");
   }
}
```

Compilation:

Use the Java compiler (javac) to convert the .java file to a .class file (bytecode).

Command: javac HelloWorld.java

→ Creates: HelloWorld.class

Execution:

Run the compiled bytecode using the Java interpreter (java command).

Command: java HelloWorld

→ Output:

Hello, World!

Steps in Compiling and Running Java Code

Step	Description
1. **Write**	Create a .java file using a text editor or IDE.
2. **Compile**	Use javac to compile the file into bytecode (.class).
3. **Load**	JVM loads the .class file at runtime.
4. **Verify**	JVM verifies the bytecode for security and correctness.
5. **Execute**	JVM interprets or uses JIT to execute bytecode.

➢Summary of Key Concepts:

- Java is a **versatile, scalable,** and **secure language** that dominates industries such as web, enterprise, mobile, embedded systems, and cloud computing, making it one of the most in-demand technologies in the software world.
- Java source code (.java) → Compiled into bytecode (.class) by javac.
- Bytecode is **platform-independent**, run by JVM on any OS.
- Java follows a **Write Once, Run Anywhere** philosophy.
- JVM lifecycle includes: **class loading**, **bytecode verification**, and **execution**.

➢References:

1. **Oracle Official Java Documentation**
   - https://docs.oracle.com/javase/tutorial/
2. **"Java: The Complete Reference" by Herbert Schildt**
   - McGraw-Hill Education, Latest Edition
3. **GeeksforGeeks – Java Programming Language**
   - https://www.geeksforgeeks.org/java/
4. **Java Platform Overview – Oracle**
   - https://docs.oracle.com/javase/8/docs/technotes/guides/
5. **TutorialsPoint – Java Programming**
   - https://www.tutorialspoint.com/java/

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

https://paruluniversity.ac.in/